

//Wentzel
//Miguel
//4478677
//CSC212 2026 Practical 2 Term 3

Question 3 : Computational Efficiency

1. Greedy Implementation

The Greedy implementation first sorts the trips according to their finishing time. Sorting n trips has a worst-case time complexity of $O(n \log n)$.

After sorting, a single for-loop examines each trip once. This loop takes $O(n)$ time. The compatibility test inside the loop uses constant-time comparisons and arithmetic, so it takes $O(1)$ for each trip.

Combining the sorting and the linear scan gives $O(n \log n) + O(n)$. The dominant term is $O(n \log n)$, so the overall worst-case runtime of the Greedy implementation is:

$O(n \log n)$

2. Dynamic Programming Implementation

The Dynamic Programming implementation also sorts the trips first, which takes $O(n \log n)$ time.

Next, the code calculates the p array. For each trip, an inner loop searches backwards through previous trips to find a compatible trip. In the worst case, this can examine up to n previous trips for each of n trips. Therefore, this nested-loop section takes $O(n^2)$ time.

The dp array is then filled using one loop. Each array lookup, such as $dp[p[i - 1] + 1]$, takes $O(1)$ time because array access by index is constant time. Filling the DP array therefore takes $O(n)$ time.

The reconstruction of the selected trips uses a while-loop that moves backwards through the DP results and takes $O(n)$ time in the worst case. Reversing the selected list is also $O(n)$.

The dominant term is the $O(n^2)$ nested-loop operation. Therefore, the overall worst-case runtime of the Dynamic Programming implementation is:

$O(n^2)$

3. Summary

Implementation	Main Operations	Big-O Runtime
----------------	-----------------	---------------

Greedy	Sorting + single loop	$O(n \log n)$
Dynamic Programming	Sorting + nested compatibility loop + DP array	$O(n^2)$

4. Conclusion

The Greedy implementation has an overall worst-case runtime of $O(n \log n)$, mainly because of sorting. The Dynamic Programming implementation has an overall worst-case runtime of $O(n^2)$, mainly because of the nested loops used to determine compatible trips.

Dynamic-array lookups are $O(1)$ and therefore do not increase the overall order of the algorithm.